

Arquitetura e Organização de Computadores I

Prof. Anderson

19 de junho de 2026

Conteúdo

1	Apresentação da Matéria	2
2	Conceitos Básicos	2
2.1	Função vs Estrutura	2
2.2	ENIAC – Eletronic Numerical Integrator and Calculator	2
2.3	John Von Neumann	3
3	Estruturas de Interconexão do Computador	4
3.1	Computador IAS	4
3.2	Programação em hardware	5
3.2.1	Ciclo de Instrução	5
3.2.2	Interrupção	6
3.3	Conexões	6
3.3.1	Conexão da Memória Principal	7
3.3.2	Conexão de Entrada/Saída	7
3.3.3	Conexão da CPU	7
4	Tipos de Transferências	8
4.1	Barramento (BUS)	8
4.1.1	Barramento de Dados	9
4.1.2	Barramento de Endereços	9
4.1.3	Barramento de controle	9
4.1.4	Aspectos fisicos dos barramentos	9
4.1.5	Barramento Único	10
4.1.6	Classificação de Barramentos	10
4.1.7	Arbitragem do Barramento	10
4.1.8	Gargalo de von Neumann	11
5	Aritmética Computacional	11
5.1	Unidade Lógica Aritimética(ULA)	11
5.1.1	Números Inteiros	12
6	Conjunto de Instruções	16

OBS.: As primeiras anotações serão da aula do professor Calvo, visto que o professor Anderson ainda está em viagem.

1 Apresentação da Matéria

2 Conceitos Básicos

- Arquitetura são os atributos visíveis ao programador
 - Conjuntos de instruções, números de bits usados para representação de dados, mecanismos de E/S, técnicas de endereçamento.
 - Exemplo: Existe uma instrução de multiplicação?
- Organização é como os recursos são implementados.
 - Sinais de controle, interfaces, tecnologia de memória
 - Exemplo: Existe uma unidade de multiplicação no hardware ou ela é feita pela adição repetitiva

Exemplo de Decisão de Projeto

- Questão: Presença ou não da instrução de multiplicação.
- A decisão deve ser tomada de acordo com:
 - A frequência do uso da instrução
 - Velocidade relativa das duas técnicas
 - Custo e tamanho físico de uma unidade de multiplicação especial

2.1 Função vs Estrutura

- Função – É a operação individual de cada componente como parte da estrutura
 - As funções do computador são:
 - * Processamento de dados
 - * Armazenamento de dados
 - * Movimentação de dados
 - * Controle
- Estrutura – É o modo como os componentes são interrelacionados

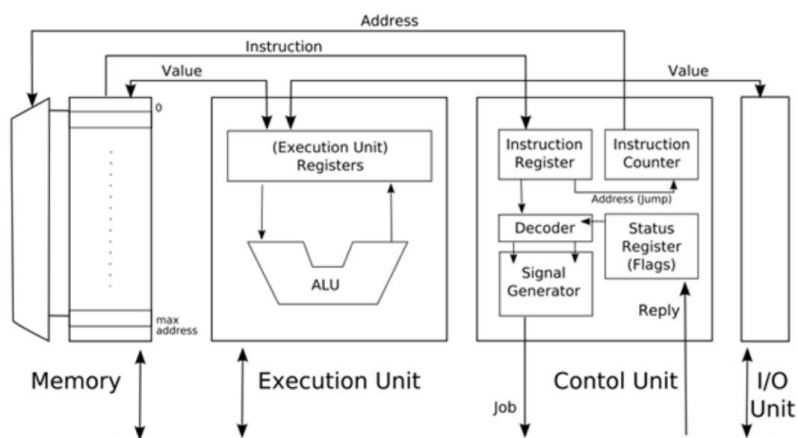
2.2 ENIAC – Eletronic Numerical Integrator and Calculator

- Sediado na Universidade da Pensilvânia
- É um computador de propósito geral construído entre 1943 e 1946
- Realizava cálculos de balística na WW2
- Consumiu uma pequena fortuna de \$ 500.000 da época
- Ocupava uma área de 150 m² e pesava 30 toneladas

- Era acionado por um motor equibalente a dois potentes motores de carros de quatro cilindros, enquanto um enorme ventilador refrigerava o calor produzido pelas válvulas
- Consumia 150.000 watts ao produzir o calor equivalente a 50 aquecedores domésticos
- Possuía 20 registradores, cada um com capacidade para armazenar um número decimal de 10 algarismos
- A programação era feita através de fios e pinos
- Executava 5000 adições/subtrações ou 300 multiplicações por segundo
- Para programar levava 1 ou 2 dias
- Uma grande limitação era a capacidade de armazenamento de dados

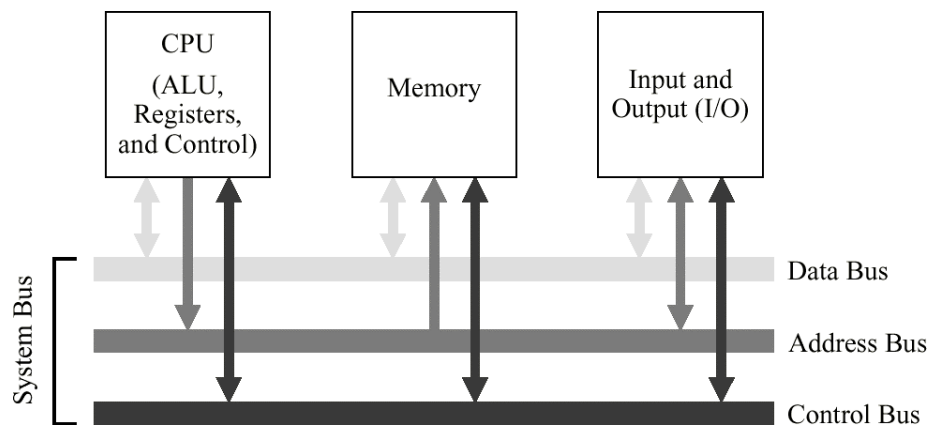
2.3 John Von Neumann

- Consultor do projeto ENIAC
- Criou o conceito de "programa armazenado"
- Criou o conceito de operações com número binários
- Desenvolveu a lógica dos circuitos
- Memória principal armazenando programas e dados
- Unidade de controle interpretando e executando instruções da memória
- Criador do modelo de Arquitetura de Von Neumann
 - Esta arquitetura foi utilizada no EDSAC, o primeiro computador de programa armazenado



- O modelo de Von Neumann possui cinco componentes principais:
 - Unidade de Entrada
 - Unidade de Saída
 - Unidade Lógica Aritmética (ULA)

- Unidade de Memória
- Unidade de Controle (UC)
- Memória: 4096 palavras
- Cada palavra possuía 40 bits
 - Duas instruções de 20 bits
 - Um dado de 40 bits
- ULA + UC = CPU
- A Unidade de Controle possuía um registrador de 40 bits (acumulador)
- A comunicação entre os componentes é realizada através de um caminho compartilhado chamado barramento de sistema (bus)



3 Estruturas de Interconexão do Computador

3.1 Computador IAS

- O primeiro computador eletrônico, construído
- Registradores do IAS

Registrador	Tamanho (bits)	Função
Contador de Programa (PC)	12	Armazena o endereço da próxima instrução
Acumulador (AC)	40	Armazenamento temporário de dados
Multiplier quotient (MQ)	40	Armazenamento temporário de dados
Buffer de dados/instruções (MBR)	40	Dados de leitura/escrita de memória
Register de instrução (IR)	8	Armazena opcode de uma instrução
Endereço de memória (MAR)	12	Armazena endereço de memória de uma instrução

- Se voce deslocar um numero de bit uma casa pra esquerda, exemplo, 01010 para 10100, é a mesma coisa que multiplicar por 2, se mover para a direita o contrario é verdadeiro.
- É possivel colocar um nome para o endereço de memória que começa a função, para que, por exemplo, no jump, seja possivel ir para esse endereço
- Arquitetura de Von Neumann
 - A Arquitetura de Von Neumann é baseada em 3 conceitos:
 - 1) Dados e instruções são armazenados em uma mesma memória
 - 2) O conteudo da memoria é endereçavel sem considerar o tipo de dado
 - 3) A execução ocorre sequencialmente de uma instrução para outra (exceto que ocorra uma modificação)

3.2 Programação em hardware

- O armazenamento e operação de dados binários e logicos sao realizados a partir da combinação de componentes logicos
- A combinação especifica de componentes logicos pode ser feita para realizar calculos especificos
- Conexão de componentes lógicos → Programa hardwired
- A combinação de componentes logicos pode ser feita para a execução de várias funções (uso geral)
 - Sinais de controle são aplicados ao hardware

3.2.1 Ciclo de Instrução

- Um ciclo de instrução é composto por duas etapas
 - Ciclo de busca
 - Ciclo de execução
- Ciclo de busca
 - Contador de Programa (PC) mantem endereço da proxima instrução a buscar
 - Processador busca a instrução do local de memoria apontado pelo PC
 - Incrementar PC – A menos que seja informado de outra forma
 - Instrução carregada no Registrador de Instrução (IR)
- Ciclo de Execução / instrução
 - Processador interpreta instrução e realiza as ações exigidas
 - Categoria das ações:
 - * Processador – memória
 - * Processador – E/S
 - * Processamento de dados
 - * Controle

3.2.2 Interrupção

- Mecanismo pelo qual módulos (como E/S) podem interromper a sequência de processamento normal
- Interrupções podem ser ocasionadas por:
 - Falha de algum programa: divisão por zero, acesso à espaço de memória inválido, overflow
 - Timer: sinal para indicar o instante da execução de uma função específica
 - E/S: sinalizar o termino de uma operação ou ocorrência de erros
 - Falha de hardware: falta de energia
- Ciclo de Interrupção
 - Adicionado ao ciclo de instrução
 - Processador verifica Interrupção
 - * Indicado por um sinal de Interrupção
 - Se não houver interrupção, busca próxima instrução
 - Se houver interrupção pendente:
 - * Suspende execução do programa atual
 - * Salva contexto
 - * Define PC para endereço inicial da rotina de tratamento de interrupção
 - * Interrupção do processo
 - * Restaura contexto e continua programa interrompido
- Múltiplas interrupções
 - Desativar Interrupções
 - * Processador ignorara outras interrupções enquanto processa uma interrupção
 - * Interrupções permanecem pendentes e são verificadas após a primeira interrupção ter sido processada
 - * Interrupções tratadas em sequência enquanto ocorrem
 - Definir prioridades
 - * Interrupções de baixa prioridade podem ser interrompidas por interrupções de prioridade mais alta
 - * Quando a interrupção de maior prioridade tiver sido processada, processador retorna à interrupção anterior

3.3 Conexões

- Todas as unidades de um sistema computacional precisam estar conectadas
- Tipos diferentes de conexão para tipos diferentes de módulos:
 - Memória
 - Módulo de E/S
 - CPU

3.3.1 Conexão da Memória Principal

- Tipicamente, a memória principal é composta de N palavras de mesmo tamanho
- Receve e envia palavras:
 - Endereços (de posições)
 - Dados
- Recebe sinais de controle
 - Leitura
 - Escrita

3.3.2 Conexão de Entrada/Saída

- Similar à memória, do ponto de vista da CPU
- Cada interce (porta) é identificada por um endereço
- Operação de Saída
 - Recebe dados do computador
 - Envia dados para a interface do dispositivo externo
- Operação de Entrada
 - Recebe dados da interface de um dispositivo externo
 - Envia dados para o computador
- Recebe sinais de controle do computador
- Envia sinais de controle para os dispositivos externos
 - Ex.: Ligar motor do disco rígido
- Recebe endereços do computador
 - Ex.: Número da porta para identificar periférico
- Envia sinais de interrupção (controle)

3.3.3 Conexão da CPU

- Lê instruções e dados
- Escreve dados (após processamento)
- Envia sinais de endereçamento para outras unidades
- Envia sinais de controle para outras unidades
- Recebe (e processa) as interrupções

4 Tipos de Transferências

- Memória principal para processador
imagem
- Processador para memória principal
imagem
- Subsistema de E/S para processador
imagem
- Processador para subsistema de E/S
imagem
- Entre memória e subsistemas de E/S (DMA) - Opcional

4.1 Barramento (BUS)

- Há diversas maneiras de se projetar um sistema de interconexão
- Mais comuns:
 - Barramento separado em linhas de dados, endereço de controle
 - Barramento compartilhado (multiplexado) para vários tipos de informação (i.e.: dados e endereços)
- Meio de comunicação entre dois ou mais dispositivos
- Sinais colocados em um barramento são recebidos por todos os dispositivos (broadcast)
- Apenas um módulo pode escrever em um barramento em um dado instante
- Geralmente é agrupado:
 - Vários canais de comunicação em um só barramento (barramento paralelo)
 - Barramento de dados de 32 bits é um conjunto de 32 canais de 1 bit
- Nos modelos não se representam as conexões para a fonte de tensão
imagem
- No diagrama acima o barramento está dividido em:
 - Barramento de dados
 - Barramento de endereço
 - Barramento de controle

4.1.1 Barramento de Dados

- Transmite dados
 - Observe que não há distinção entre "dado" e instrução neste nível (fora da CPU)
- Largura do barramento é um fator determinante no desempenho (via da regra, quanto mais largo, melhor)
- Largura de barramento de dados comuns:
 - 8 bits (Intel 8088)
 - 16 bits (Intel 8086)
 - 32 bits (Intel Pentium)
 - 64 bits (Intel Xeon – Core2Duo/Quad – Core i3/5/7)

4.1.2 Barramento de Endereços

- Usado para identificar a fonte e o destino dos dados que trafegam no barramento de dados
- Ex.: CPU precisa ler uma instrução (dado) de uma determinada posição na memória
- Largura do barramento de endereço determina a capacidade de memória máxima do sistema
 - Ex.: Z-80 tem um barramento de endereço de 16 bits.
Memória máxima: $2^{16} = 65.536$ posições de memória.

4.1.3 Barramento de controle

- Controle e temporização das informações
 - Sinais de leitura e escrita para a memória principal para dispositivos de E/S.
 - Requisições de interrupção
 - Sinais de controle gerais, tais como:
 - * Requisição de uso do barramento por um módulo
 - * Término de uso do barramento
 - * Inicialização (reset) de dispositivos, etc.

4.1.4 Aspectos físicos dos barramentos

- Conectores de borda de placa de expansão (Ex.: PCI)
- Cabos paralelos
- Linhas paralelas em placas de circuito impresso

4.1.5 Barramento Único

- Apesar de ser mais simples de implementar, há desvantagens de se usar um unico barramento de sistema
 - Condutores longos: tempo maior de propagação
 - Muitos dispositivos ligados no mesmo meio de transmissão significa que a ordenação do uso do barramento pode afetar o desempenho geral do sistema
 - E se houver dispositivos com diferentes velocidades de transmissão ligados ao barramento?
 - * Taxa de transferência total de dados é limitada à taxa de transferência no barramento do sistema
 - Solução: Usar múltiplos barramentos

4.1.6 Classificação de Barramentos

- Dedicado
 - Linhas separadas para dados e endereços
- Multiplexado
 - Linhas compartilhadas
 - Necessita de mais uma linha de controle, para indicar se há dados ou endereços trafegando no barramento naquele instante
 - Vantagem: Necessita de menos condutores
 - Desvantagens:
 - * Controle mais complexo
 - * Pode diminuir o desempenho (pois compartilha um só canal para endereços e dados)

4.1.7 Arbitragem do Barramento

- Apenas um módulo pode controlar o barramento por vez (apesar de mais de um módulo poder ler o que está sendo transmitido)
- O que acontece se mais de um módulo solicitar o controle do barramento?
- Claramente, precisamos estabelecer regras para ceder o controle do barramento: Arbitragem do Barramento
- As regras definem mestre (que controla) e escravo(s) (que é/ são controlados)
- A arbitragem pode ser centralizada ou distribuida
- Tipos de Arbitragem
 - Centralizada
 - * Somente um módulo controla o acesso ao barramento

- Controlador de barramento ou árbitro decide quem poderá controlar o barramento
- * Pode ser parte da CPU, ou um dispositivo dedicado a essa tarefa
- Distribuída
 - * Cada módulo pode decidir controlar o barramento
 - * Necessária lógica de controle em todos os módulos

4.1.8 Gargalo de von Neumann

- Refere-se ao tráfego nos barramentos do computador
 - Vai endereço da instrução, volta instrução
 - Vão endereços dos operandos
 - Vão e voltam operandos
- Para eliminar gargalo: Diminuir tráfego de informações
 - Manter informações na CPU → Inclusão de registradores
 - Diminuir tamanho (em bits) das informações transferidas

5 Aritmética Computacional

5.1 Unidade Lógica Aritimética(ULA)

- É a parte do computador que de fato executa as operações aritméticas e lógicas sobre os dados
 - Tudo o mais no computador existe para atender a essa unidade
 - Todos os outros elementos do computador são usados para:
 - Trazer os dados a serem processados pela ULA
 - Receber os resultados das operações efetuadas
 - Constitui o núcleo ou a essência do computador
 - É baseada em dispositivos lógicos digitais simples, capazes de armazenar dígitos binários e efetuar operações simples de lógica booleana
 - Os dados são fornecidos à ULA em registradores e os resultados de uma operação são armazenados também em registradores
 - A ULA pode também ativar bits especiais (flags) para indicar o resultado de uma operação
- imagem
- A aritmética computacional geralmente opera com dois tipos de números muito diferentes:
 - Números inteiros

- Numeros de ponto flutuante
- Em ambos os casos, a escolha da representação é uma questão crucial de projeto

5.1.1 Números Inteiros

- Representação de números inteiros
 - No sistema de números binários, é possível representar números arbitrários usando os dígitos zero e um

$$-1101,101_2 = -13,625_{10}$$

- Para armazenar esses números no computador, não é possível usar os sinais de menos e a vírgula

- Apenas os dígitos binários (0, 1) podem ser usados
- Representação sinal-magnitude
 - Utiliza o bit mais significativo da palavra como um bit de sinal
 - * Se o bit mais significativo for 0, o número será positivo
 - * Se for 1, o número será negativo
 - É a forma mais simples de representação

$$+18_{10} = 00010010$$

$$-18_{10} = 10010010$$

- Algumas desvantagens dessa forma de representação
 - * Numa operação de adição e subtração é preciso considerar:
 - A magnitude do número
 - O sinal dos dois operandos
 - * Existem duas representações de 0:

$$+0_{10} = 00000000$$

$$-0_{10} = 10000000$$

- Representação em complemento de dois
 - Também usa o bit mais significativo como bit de sinal
 - Os números são escritos da seguinte forma:
 - * Positivos: Sua magnitude é representada na sua forma binária direta, e um bit de sinal 0 é colocado em sua frente
 - (bit 0) + o número em binário
 - Exemplos:

$$0001(+1)$$

$$0100(+4)$$

$$0111(+7)$$

- * Negativos: Sua magnitude é representada na forma de complemento a 2 (um bit de sinal acaba ficando em sua frente)
 - Para um número binário de n bits, o peso do bit mais significativo é -2^{n-1}

$$A = -2^{n-1}a_{n-1} + \sum_{i=0}^{n-2} 2^i a_i$$

- Regra para formar a negação:
 1. Inverte-se os bits do número em binário
 - * 0100 $\rightarrow$ Invertendo temos $\rightarrow$ 1011
 2. Soma-se um ao valor invertido:
 - * 1011 + 0001 = 1100
- O zero é tratado com um numero positivo
- Só existe uma representação de zero
 - * Realizando o complemento de dois de zero
imagem
- É possível representar $2^{n-1} - 1$ numeros positivos
- É possível representar 2^{n-1} numeros negativos
- A representação em complemento de dois parece "pouco natural"
- Porém ela torna mais simples a implementação das operações 36aritméticas mais importantes (adição e subtração)

Ciclo de Instruções

- Gerenciado pela Unidade de Controle
- Dividido em 4 etapas

–

Troca de Contexto

- Salva o estado da operação

Maneiras de otimizar a o ciclo de instruções

- Pipeline
- Cache

– Hierarquia de memória:

- * Disco
- * RAM (Memória de Acesso Aleatório)
- * Cache
- * Registradores

– Ao descer na hierarquia, a memória é mais rápida mas com tamanho mais escasso – Latência é praticamente nula.

- O barramento da memória RAM com o processador é um gargalo e possui uma latência L
- A memória cache foi criada com o intuito de reduzir esse gargalo, ao invés de buscar apenas uma instrução, busca um bloco com elas, e guarda esse bloco na memória.
- É possível ter níveis diferentes de cache, localizações diferentes, e organizações diferentes.

Cache

- Armazena dados e instruções
- As instruções tem tempos fixos para a execução de acordo com os circuitos eletrônicos do processador
- A memória cache não deixa as instruções mais rápidas, deixa o programa mais eficiente
- Localidade:

- Na memória cache temos dois tipos de localidade:

1. Temporal – Uma região de memória de um determinado dado será acessado com uma constância maior

* Ex.:

```
for (int i = 10; i <= 10000; i++) {
    soma += 1;
}
```

* Nesse caso, `soma` é uma variável temporal

2. Espacial – Os dados próximos ao dado que foi acessado anteriormente, serão acessados também

* Ex.:

```
for (int i = 10; i <= 10000; i++) {
    v[i] = i;
}
```

* Nesse caso, `v[i]` é uma variável espacial

- Quando o programa possui uma função JUMP, a memória cache busca os dados aos arredores do endereço do JUMP, porém, não é sempre que a cache acessa a memória correta

* Falha (miss) – Quando o dado ou instrução que o processador precisa não está na Cache

* Acerto (hit) – Quando o dado ou instrução que o processador precisa está na Cache

- Fluxo
- Hierarquia

- Processador → Cache N1 → Cache N2 → Cache N3 → RAM

- Organização
 - Organizada em linhas, onde cada linha é um bloco da memória
 - Atualmente as máquinas usam em média 3 níveis de memória cache, onde:
 1. O nível um normalmente é dividido em dados e instruções
 2. O nível dois é unificado, onde não tem um espaço especial para ou dados ou instruções
 3. O nível três também é unificado
 - A cache não possui palavras, ela possui linhas, que normalmente contem um numero maior de palavras
- Políticas de Substituição:
 - LRU – Least Recently Used
 - * Retira a linha que está a mais tempo sem ser utilizada
 - FIFO – First in, First out
 - * Retira a linha que está na memória a mais tempo
 - Random
 - LFU – Less Frequently Used
 - * Retira a linha que foi usada com a menor frequencia dentro do programa
 - Dependendo da localidade, esses algoritmos podem ser mais e menos eficientes, aumentando os hits ou misses.
 - gem5 – Fazer testes, usando o modo SE
- Estratégias de escritas:
 - a. Write Back
 - Mais complexa, porém tem custo de memória mais baixo
 - Cada bloco só é armazenado por linha de cache
 - b. Write Through
 - Mais simples mas com custo de memória mais alto.
 -
- Estratégias de Mapeamento de Cache
 - Decide em qual linha um bloco de memória será armazenado na cache
 - Temos 3 tipos de mapeamento
 1. Direto
 - * Um bloco de memória é sempre armazenado na mesma linha
 - * Utiliza de um endereço:

Tag	KBits	Palavra
-----	-------	---------

- * Tag indica
- * Os Kbits indicam em qual linha de cache o bloco será armazenado

- * Palavra indica qual palavra do bloco de memória será acessado
- * No mapeamento direto não é útil nenhum tipo de política de substituição, já que o bloco de memória sempre cai na mesma linha
- * Dentro do mapeamento direto, dois endereços podem acabar em uma mesma linha, e retirar outro endereço que já estava lá e que é útil ao programa.
 - Isso pode gerar um problema denominado **Trashing**, onde a eficiência de um programa pode cair devido a constante troca nas linhas.
 - Esse problema pode ser resolvido utilizando do mapeamento Associativo

2. Associativo

- * Um bloco é associado a uma linha qualquer da Cache
- * É necessário realizar buscas para encontrar o bloco

Tag	Palavra
-----	---------

- * No associativo, a **tag** é maior que por mapeamento direto

3. Associativo por conjunto

- * Junção do direto e do associativo
- * As linhas são organizadas por conjunto também
- * Um bloco sempre cai em determinado conjunto de linhas, porém pode cair em qualquer linha dentro desse conjunto

Tag	Conjunto	Palavra
-----	----------	---------

Tamanho da memória = 2^n bytes

Tamanho da cache = 2^s linhas

Tamanho da linha = 2^w bytes

Um endereço possui n bits, onde armazena um offset de w bits, um índice para uma linha de s bytes, e uma tag de $n - s - w$ bits

Para encontrar o endereço, ele procura pela linha dada por s , onde ao encontrar a linha, ele faz vai até o bloco w de memória, porém, como podem

Em uma instrução, quando coloca o endereço em parenteses, quer dizer que, o dado que será usado na operação está dentro do endereço do registrador entre parenteses

6 Conjunto de Instruções

- ISA
- CISC e RISC
 - CISC – Complexo
 - * Intel (CISC)
 - * Complexidade nas instruções devido ao acesso de memória custoso.

- * Uma instrução pode ter uma ou mais tarefas
- * Um dos parâmetros pode ser um dos locais da memória
 - Assembly – Linguagem de Programação
 - Assembler – Montagem da linguagem
- * Unidade de Controle microprogramada em software – Uma memória contém o microprograma e ao receber uma instrução, envia para essa memória e envia para o banco de registradores e para a ULA realizar as operações
- RISC – Reduzido
 - * ARM (RISC)
 - * Simplificar as instruções
 - * Possui apenas duas instruções para acesso de memória
 - * Unidade de Controle microprogramada em hardware –
- x86 - 64 – As instruções possuem 64 bits
- 16 registradores – Registradores de Propósito Geral

Registradores de Propósito Geral podem ser usados no código, já os de Propósito Específico, não é possível usar para coisas além de leitura

 - * AX – 16 bits
 - * BX – 16 bits
 - * CX – 16 bits
 - * DX – 16 bits
 - * SI – 16 bits
 - * DI – 16 bits
 - * SP – 16 bits
 - * BP – 16 bits
 - * R8 – R15
- Em arquiteturas de 16 bits, só existem os 8 primeiros registradores, para transformar em uma arquitetura de 32 bits basta usar o prefixo E dos registradores, que transforma os registradores em 32 bits, e para uma arquitetura de 64 bits, basta colocar o prefixo R, nessa arquitetura também existe os 8 registradores do R8 ao R15
- AX, BX, CX e DX são utilizados para instruções aritméticas.
 - * Algumas convenções são, CX é utilizado para contadores, AX é utilizado como valor de retorno.
- SI e DI são registradores usados para indexar memória, normalmente o SI possui o endereço e o DI possui o dado
- SP e BP são utilizados para o conceito de pilha, próximo ao que é visto em estrutura de dados, serve para conter informações da função corrente como parâmetros e variáveis locais.
 - * Nesse conceito de pilha, não é necessário desempilhar para verificar os elementos de fora do topo da pilha, BP (Base Pointer) é utilizado para fazer essa movimentação
- Registradores específicos

- Formato – Uma instrução é normalmente uma palavra formada por x bits, dentro da palavra existe um opcode, e diversos outros endereços
- Tipos de Endereço:
 - Imediato – Já possui o valor que será processado.
 - Registrador – Possui um registrador.
 - Direto – Possui um indicador para um endereço de memória que possui o dado.
 - Indireto – Possui um indicador para um endereço em um registrador, que indica outro endereço onde está o dado.
 - Deslocamento – Endereço indireto, porem com uma soma ao valor de memória
 - Deslocamento e escala – Endereço indireto, porem com uma escala (Soma se ao endereço $x \times y$)
 - Rip-relativo – Endereço de Deslocamento porem somente para arquiteturas 64 bits
- Instruções
 - A
 - * add
 - * sub
 - * div
 - * mul
 - L
 - * sll
 - * shr
 - * not
 - * and
 - * or
 - * xor
 - * ...
 - C
 - * Jump
 - * J?
 - e
 - g
 - l
 - ge
 - le
 - z
 - ...
 - M
 - * mov 0, D

```

mov 0, ecx;
add 1, ecx : inicio;
cmp ecx, 100;
je fim;
mov x, eax;
add x, eax;
add ecx, eax;
jump inicio;
mov eax, [0x1092] : fim;
  sintaxe AS, cartão de referencia assembler
  diretivas:

```

- .section
 - .data – Onde é declarado as variaveis (variavel : tipo da variavel e valor da variavel)
 - .bss – Onde é declarado variáveis não iniciados
 - .text – Onde é escrito o codigo (funções começam com _)
 - * O codigo no assembly sempre começa com um **_start** global
- .glob – indica uma função que pode ser usada no arquivo todo
- .ascii (asciz – tem um \0 no final, ascii não coloca o terminador automaticamente)
- .byte – 1
- .word – 2
- .long – 4
- .equ – 8

sintaxe:

- AT&T
- Intel

bss – área estática

heap – área da memoria alocada pelo usuário (malloc)

registrador – % nome do registrador

numero imediato – \$ numero

mov, O, D

add O, D

sub O, D

cmp O, D

jmp R

j? R

call R – Salva o endereço de retorno

ret – Captura o endereço de retorno

int N syscall – interrupção

- `int $0x80` – indica alguma interrupção

cada funcionalidade que eu desejo deve ser armazenada no registrador `eax`
após uma função pode colocar um indicador do tipo da variável (ex.: para mover um
`long` ficaria `movl`)

final do código (`exit`):

```
movl $1, %eax
movl $0, %ebx
int $0x80
```

como compila:

```
$ as teste.s -o teste.o
```

```
$ ld teste.o -o teste
```

Exemplos

1. ler 2 números e imprimir a soma
2. fibonacci
3. multiplicação de matrizes